AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Drzewo BST

Autor:
Wiktor Matusik
Mateusz Kowalczyk

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

2. Analiza problemu **5**

2.1. Czym jest Drzewo BST 5
2.2. Operacje i sposoby przechodzenia przez drzewo 6
2.3. Zastosowania drzew BST 7
2.4. Sposób działania na przykładzie 8
2.5. Opis użytych narzędzi 8
2.6. Sposób działania programu i algorytmu 8

3. Projektowanie **10**

3.1. Wykorzystywane narzędzia 10
3.2. Konfiguracja kompilatora 11
3.3. Schemat programu 11
3.4. Sposób użycia Git oraz GitHub 12

4. Implementacja **24**

4.1. Implementacja algorytmu 24
4.2. Ciekawe fragmenty kodu 24
4.3. Opis powstałych wyników 26

5. Wnioski **28**

Literatura **29**

Spis rysunków **30**

Spis tabel **31**

Spis listingów **32**

1. Ogólne określenie wymagań

Napisz program „drzewo BST" działającej w kontenerze STL w języku C++. Drzewo winno być zaimplementowana w klasie. Funkcjonalność (metod) drzewa:

- Dodaj element,
- Usuń element,
- Usuń całe drzewo,
- Szukaj drogi do podanego elementu,
- Wyświetl drzewo graficznie na ekranie, użytkownik wybiera metodę podczas wyświetlania (metody preorder, inorder, postorder) [oprogramuj wszystkie trzy],
- Zapis do pliku tekstowego wygenerowanego drzewa,

W drugiej klasie należy zaimplementować (metody) zapis do pliku i odczyt z pliku utworzonego drzewa BTS (plik musi być zapisany binarnie). Oprócz tego powinien mieć możliwość wczytania pliku tekstowego z cyframi, co daje to możliwość zbudowania drzewa. Program powinien posiadać możliwość wczytania pliku z liczbami do drzewa pustego lub istniejącego.

Funkcja main powinna wyświetlać menu z opcjami drzewa oraz odczytu i zapisu pliku. Program czeka na wybranie opcji. Nie zapomnij ustawić wyjścia z programu aby program można było poprawnie zamknąć.

Wszystkie utworzone klasy mają być zaimplementowane w oddzielnych plikach. Funkcja main także powinna być w osobnym pliku.

Celem zadania jest poznanie możliwości GitHuba w pracy nad projektem grupowym. Konto na GitHub jest utworzone. Należy utworzyć nowy projekt, który jest realizowany w dwuosobowych grupach.

Przy oddawaniu projektu należy zaprezentować:

- Co najmniej 5 commit'ów (każdej osoby),
- Najpierw jedna osoba tworzy gałąź i po kilku comitach ją scala. Po scaleniu druga osoba tworzy gałąź i po kilku comitach ją scala do głównej gałęzi ,
- Obie osoby w grupie mają utworzyć nowe gałęzie (w jednym punkcie obie osoby zaczynają pracę równoległą), a po kilku comitach wykonują scalenie do głównej gałęzi,

- Co najmniej 6 konfliktów, które należy rozwiązać (3 jedna osoba, 3 druga osoba) przy scalaniu gałęzi (w wcześniejszych punktach),

Przy pracy równoległej, a następnie scalaniu można używać wtyczki GitHub zainstalowanej do Visual Studio Code lub Visual Studio. Następnie sprawdźcie jak wygląda plik po scalaniu, który jest na GitHubie.

Dodatkowo do projektu należy dołączyć dokumentację w Latex z zastosowaniem uwag, które były omawiane na poprzednim projekcie. W dokumentacji wklej obraz drzewa z comitów , który należy pobrać z GitHuba oraz wygenerować dokumentacje Doxygenem. Projekt realizować bez użycia narzędzi sztucznej inteligencji takich jak ChatGPT, GitHub Copilot.

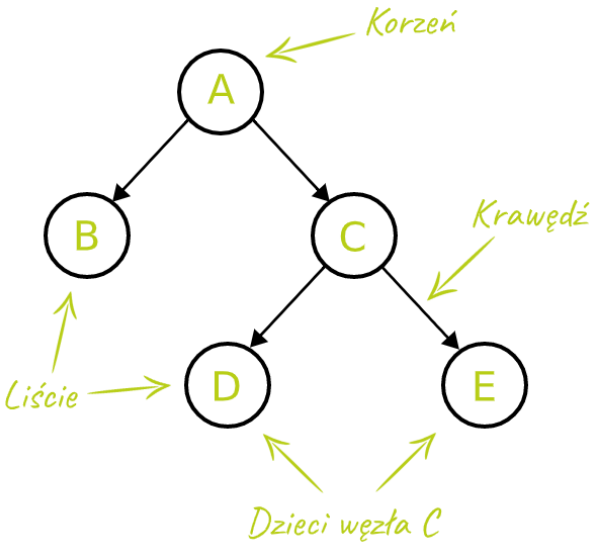
2. Analiza problemu

2.1. Czym jest Drzewo BST

Drzewo poszukiwań binarnych (ang. Binary Search Tree, BST) to rozgałęziona, hierarchiczna struktura danych, w której każdy węzeł przechowuje wartość oraz wskaźniki na co najwyżej dwóch potomków: lewego i prawego. Podstawową własnością drzewa BST jest porządek, w którym dla każdego węzła X:

- wszystkie wartości w lewym poddrzewie są mniejsze od wartości węzła X,
- wszystkie wartości w prawym poddrzewie są większe od wartości węzła X.

Dzięki tej właściwości średnia złożoność czasowa operacji wyszukiwania, dodawania oraz usuwania elementów wynosi O(log n), co czyni BST wydajną strukturą danych. Przykład drzewa BST na rysunku 2.1 (s. 5) [1].



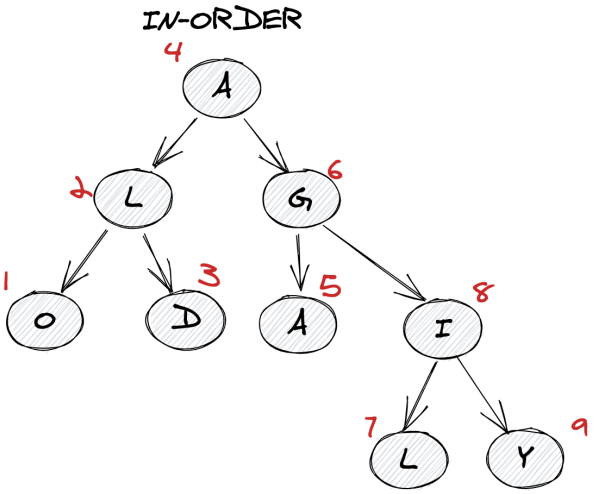
Rys. 2.1. Drzewo BST

[1] Rysunek wzięty z podanej strony[1].

2.2. Operacje i sposoby przechodzenia przez drzewo

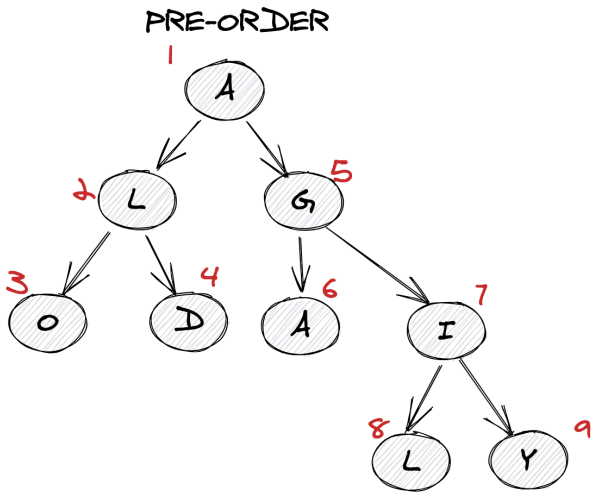
W drzewach BST istnieją trzy podstawowe metody przechodzenia i wyświetlania drzewa:

- **Inorder:** Odwiedza najpierw lewe poddrzewo, następnie węzeł główny, a na końcu prawe poddrzewo. Przeglądanie drzewa tą metodą zwraca elementy w kolejności rosnącej. Przykład na rysunku 2.2 (s. 6) [2].
- **Preorder:** Odwiedza najpierw węzeł główny, potem lewe poddrzewo, a na końcu prawe poddrzewo. Używane m.in. do kopiowania drzewa lub zapisu jego struktury. Przykład na rysunku 2.3 (s. 7) [3].
- **Postorder:** Odwiedza lewe poddrzewo, prawe poddrzewo, a na końcu węzeł główny. Metoda przydatna m.in. przy zwalnianiu pamięci. Przykład na rysunku 2.4 (s. 7) [4].

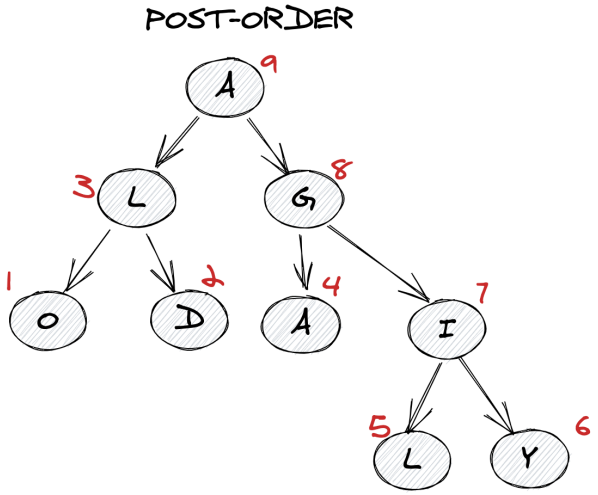


Rys. 2.2. Drzewo BST Inorder

[2]Rysunek z podanej strony[2].
[3]Rysunek z podanej strony[2].
[4]Rysunek z podanej strony[2].



Rys. 2.3. Drzewo BST Preorder



Rys. 2.4. Drzewo BST Postorder

2.3. Zastosowania drzew BST

Drzewa poszukiwań binarnych znajdują szerokie zastosowanie w informatyce m.in. w:

- Systemach baz danych i indeksowaniu danych, gdzie wymagany jest szybki dostęp do posortowanych rekordów.
- Kompilatorach i parserach do reprezentacji wyrażeń arytmetycznych i hierarchii składniowej.
- Implementacji kontenerów asocjacyjnych w bibliotekach standardowych (np. `set` i `map` w C++ opartych na zbalansowanych odmianach BST).

2.4. Sposób działania na przykładzie

Proces dodawania elementów $50, 30, 70, 20$ do początkowo pustego drzewa BST:

- **Dodanie 50:** Drzewo jest puste, więc 50 staje się korzeniem (*root*).
- **Dodanie 30:** Wartość $30 < 50$, więc zostaje umieszczona jako lewy potomek węzła 50.
- **Dodanie 70:** Wartość $70 > 50$, więc trafia jako prawy potomek korzenia 50.
- **Dodanie 20:** Wartość $20 < 50$, przechodzimy w lewo do 30, następnie $20 < 30$, więc trafia jako lewy potomek węzła 30.

2.5. Opis użytych narzędzi

- **Git i GitHub:** Rozproszony system kontroli wersji oraz platforma do współdzielenia kodu, śledzenia historii zmian i synchronizacji pracy dwuosobowej grupy.
- **Visual Studio / Visual Studio Code:** Środowiska programistyczne wspomagające pisanie kodu w języku C++ oraz integrację z systemem Git za pomocą dedykowanych wtyczek.
- **Doxygen:** Narzędzie przeznaczone do automatycznej generacji dokumentacji na podstawie specjalnych komentarzy w kodzie źródłowym.

2.6. Sposób działania programu i algorytmu

Program został zaimplementowany w języku C++ z wykorzystaniem struktur biblioteki standardowej STL, w tym kontenera `std::set<int>` do przechowywania elementów drzewa BST. Architektura opiera się na klasie `BST`, klasie pomocniczej `BSTFileManager` do operacji plikowych oraz module głównym z menu konsolowym.

- **Struktura danych i zarządzanie elementami:** Wartości w klasie `BST` są trzymane w kontenerze `std::set`, co zapewnia automatyczne porządkowanie danych. Do celów wizualizacji, przechodzenia oraz wyszukiwania ścieżki algorytm dynamicznie buduje tymczasowe, zbalansowane drzewo binarne na bazie wektora elementów.
- **Algorytmy operacyjne i przeglądania:** Klasa realizuje dodawanie, usuwanie i czyszczenie elementów, a także wyszukiwanie ścieżki do wskazanego węzła

(`findPath`). Wyświetlanie struktury oraz przechodzenie drzewa odbywa się za pomocą algorytmów rekurencyjnych w porządkach *Preorder*, *Inorder*, *Postorder* lub w czytelnej formie graficznej.

- **Obsługa plików i interfejs:** Klasa `BSTFileManager` obsługuje binarny zapis i odczyt stanu drzewa oraz import cyfr z pliku tekstowego z opcją czyszczenia lub dopisania do istniejącej struktury. Pętla główna w pliku `main.cpp` prezentuje użytkownikowi interaktywne menu konsolowe koordynujące działanie wszystkich funkcji programu.

3. Projektowanie

3.1. Wykorzystywane narzędzia

Przy realizacji projektu drzewa BST wykorzystano następujące środowisko programistyczne oraz narzędzia pomocnicze:

- **Język programowania: C++** – język obiektowy umożliwiający bezpośrednie zarządzanie pamięcią dynamiczną oraz korzystanie z kontenerów biblioteki standardowej (STL).
- **Biblioteki standardowe STL:**
  - `<set>` – wykorzystana do przechowywania danych w klasie `BST` w postaci uporządkowanego zbioru (`std::set<int>`), co ułatwia zarządzanie unikalnymi elementami.
  - `<vector>` – używana do operacji pomocniczych, przechowywania tymczasowych sekwencji oraz algorytmów budowania zbalansowanego drzewa i wyznaczania ścieżek.
  - `<fstream>` oraz `<iostream>` – odpowiedzialne za operacje wejścia/wyjścia, obsługę strumieni konsolowych oraz persystencję danych w plikach tekstowych i binarnych.
- **Kompilator: MinGW-w64** – kompilator przeznaczony dla platformy Windows, odpowiedzialny za tłumaczenie kodu źródłowego C++ do pliku wykonywalnego .exe.
- **Środowisko programistyczne (IDE): Visual Studio** – zintegrowane środowisko programistyczne wyposażone w zaawansowany edytor kodu, menedżer projektów typu Solution, wbudowany debuger oraz natywne wsparcie dla systemu kontroli wersji Git.
- **System kontroli wersji i platforma programistyczna: Git i GitHub** – narzędzia służące do śledzenia historii zmian w kodzie, pracy współdzielonej w zespole dwuosobowym, tworzenia gałęzi oraz rozwiązywania konfliktów scalania.
- **Generator dokumentacji: Doxygen** – narzędzie automatyzujące proces generowania dokumentacji technicznej z komentarzy umieszczonych w plikach nagłówkowych i źródłowych.

3.2. Konfiguracja kompilatora

Projekt zarządzany jest w ramach pliku rozwiązania (.sln) programu Visual Studio. Ustawienia kompilacji i optymalizacji kodu definiowane są bezpośrednio w panelu właściwości projektu:

- **Standard C++:** Program wykorzystuje nowoczesny standard C++20, co pozwala korzystać z najnowszych funkcji języka.
- **Ostrzeżenia kompilatora:** Poziom ostrzeżeń ustawiono na Level 3 z włączoną pełną zgodnością ze standardem, co pomaga w wykrywaniu błędów podczas pisania kodu.
- **Typ aplikacji:** Projekt jest skompilowany jako aplikacja konsolowa systemu Windows zoptymalizowana pod kątem zarządzania pamięcią.
- **Biblioteki:** Kod opiera się wyłącznie na standardowej bibliotece C++, dlatego Visual Studio automatycznie dołącza wszystkie niezbędne składniki systemowe bez potrzeby instalowania dodatkowych bibliotek.

3.3. Schemat programu

Architektura oprogramowania została zaprojektowana w oparciu o wzorzec obiektowy, z zachowaniem zasad modularności i oddzielenia logiki biznesowej od operacji wejścia-wyjścia. System składa się z następujących komponentów powiązanych w pliku rozwiązania programu Visual Studio:

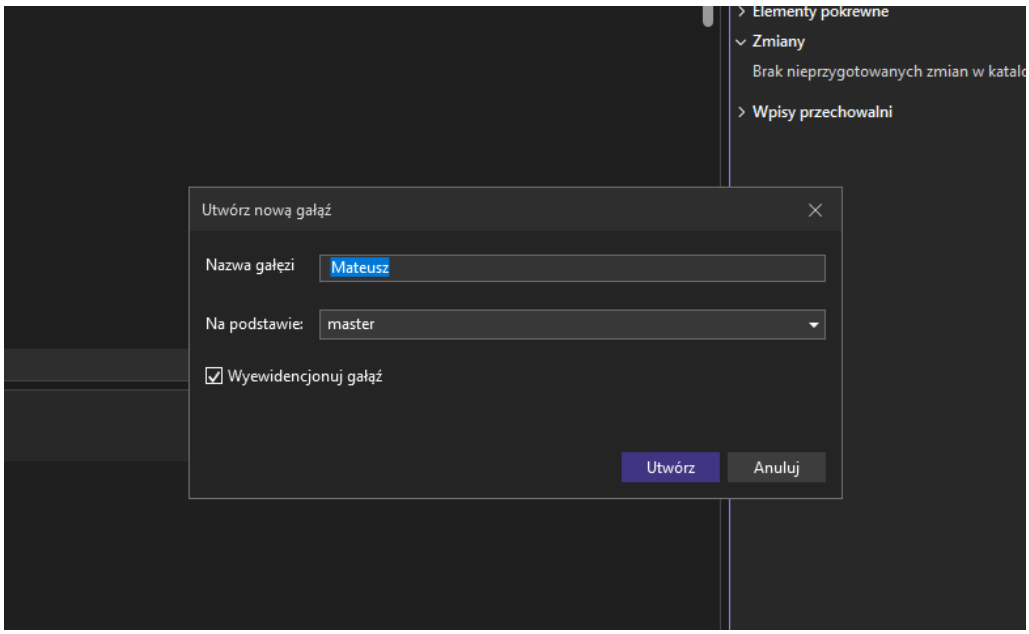
- **Klasa `BST` (`BST.h` / `BST.cpp`):** Odpowiada za główną strukturę danych. Wykorzystuje kontener biblioteki standardowej `std::set<int>` do przechowywania elementów w porządku BST. Zawiera wewnętrzną strukturę `Node` oraz implementuje metody zarządzania węzłami (`add`, `remove`, `clear`), wyszukiwania ścieżki (`findPath`) oraz graficznego i tekstowego wyświetlania drzewa w porządkach *Preorder*, *Inorder* i *Postorder*.
- **Klasa `BSTFileManager` (`BSTFileManager.h` / `BSTFileManager.cpp`):** Klasa pomocnicza odpowiedzialna za operacje trwałości danych (persystencję). Umożliwia binarny zapis i odczyt stanu drzewa oraz import sekwencji liczbowych z zewnętrznych plików tekstowych do drzewa pustego lub istniejącego.
- **Moduł główny (`main.cpp`):** Zawiera funkcję `main` oraz interaktywne pętlowe menu konsolowe, które steruje wywołaniami metod obu klas oraz zarządza cyklem życia programu.

3.4. Sposób użycia Git oraz GitHub

Praca nad projektem realizowana była w zespole dwuosobowym z wykorzystaniem rozproszonego systemu kontroli wersji Git oraz platformy GitHub, co umożliwiło efektywne współdzielenie kodu, śledzenie historii zmian oraz zarządzanie gałęziami projektu. Do obsługi repozytorium wykorzystywano zarówno wbudowane narzędzia graficzne środowiska Visual Studio (panel *Git Changes*), jak i natywny terminal poleceń.

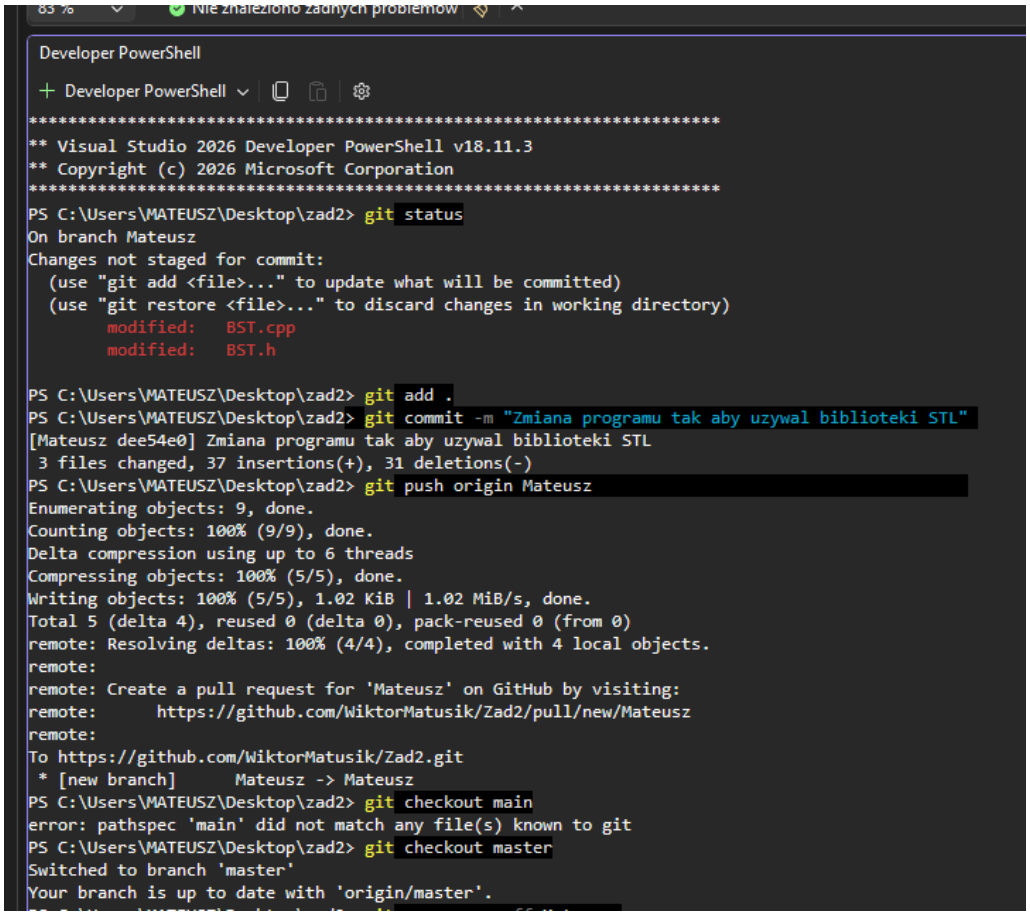
Zgodnie z wymaganiami projektowymi, proces wersjonowania obejmował pracę sekwencyjną, gałęzie równoległe oraz celowe wygenerowanie i rozwiązanie co najmniej 6 konfliktów scalania (po 3 przez każdego członka zespołu).

W pierwszej kolejności członkowie zespołu tworzyli osobne gałęzie funkcjonalne, w których wykonywali wymagane commity, a następnie scalali je z gałęzią główną (`master`). Tworzenie nowej gałęzi (np. dla użytkownika Mateusz) z poziomu interfejsu graficznego środowiska Visual Studio zaprezentowano na zrzucie ekranu 3.1 (s. 12).



Rys. 3.1. Tworzenie nowej gałęzi w interfejsie graficznym Visual Studio

Na zrzucie ekranu 3.2 (s. 13) przedstawiono proces zatwierdzania zmian i wypychania gałęzi *Mateusz* na serwer zdalny.



Rys. 3.2. Zatwierdzanie zmian i wypychanie gałęzi Mateusz

Kolejnym krokiem było przełączenie się na gałąź główną i scalenie gałęzi *Mateusz* za pomocą polecenia `git merge --no-ff`, co zilustrowano na zrzucie ekranu 3.3 (s. 14).

```
PS C:\Users\MATEUSZ\Desktop\zad2> git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
PS C:\Users\MATEUSZ\Desktop\zad2> git merge --no-ff Mateusz
Merge made by the 'ort' strategy.
 BST.cpp   | 45 ++++++
```

```
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git checkout -b Wiktor
Switched to a new branch 'Wiktor'
```

Rys. 3.5. Tworzenie nowej gałęzi przez Wiktora

Sprawdzanie statusu oraz dodawanie commitów na gałęzi Wiktora zaprezentowano na zrzucie ekranu 3.6 (s. 15).

```
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git checkout Wiktor
M       BST.cpp
M       BST.h
M       main.cpp
Switched to branch 'Wiktor'
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git status
On branch Wiktor
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   BST.cpp
        modified:   BST.h
        modified:   main.cpp

no changes added to commit (use "git add" and/or "git commit -a")
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git add .
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git commit -m "Dodanie funckji szukania sciezki do elementu"
[Wiktor 8cd035a] Dodanie funckji szukania sciezki do elementu
 3 files changed, 57 insertions(+)
```

Rys. 3.6. Sprawdzanie statusu i dodawanie commitów na gałęzi Wiktor

Wypchnięcie gałęzi Wiktora oraz jej scalenie z masterem zilustrowano na zrzucie ekranu 3.7 (s. 15).

```
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git push origin Wiktor
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 1.07 KiB | 1.07 MiB/s, done.
Total 5 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/WiktorMatusik/Zad2.git
   85086bf..8cd035a  Wiktor -> Wiktor
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git merge --no-ff Wiktor
Merge made by the 'ort' strategy.
 BST.cpp  | 36 ++++++++++++++++++++++++++++++++++++
 BST.h    |  2 ++
 main.cpp | 19 +++++++++++++++++++
 3 files changed, 57 insertions(+)
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git push origin master
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 225 bytes | 225.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/WiktorMatusik/Zad2.git
   85086bf..e9cc43b  master -> master
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2>
```

Rys. 3.7. Wypchnięcie gałęzi Wiktor i scalenie jej z masterem

W celu spełnienia wymagań pracy równoległej, obaj członkowie zespołu utworzyli nowe gałęzie robocze (np. `rownolegla-wiktor` oraz `rownolegla-Mateusz`) wy-

chodzące z tego samego punktu w historii projektu. W gałęziach tych prowadzono niezależną implementację nowych funkcjonalności.

Pobranie najnowszych zmian i utworzenie gałęzi równoległej przez Wiktora przedstawiono na zrzucie ekranu 3.8 (s. 16).

```
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git checkout master
Already on 'master'
Your branch is up to date with 'origin/master'.
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git pull origin master
From https://github.com/WiktorMatusik/Zad2
 * branch            master     -> FETCH_HEAD
Already up to date.
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git checkout -b rownolegla-wiktor
Switched to a new branch 'rownolegla-wiktor'
```

Rys. 3.8. Pobranie najnowszych zmian i tworzenie gałęzi równoległej przez Wiktora

Utworzenie gałęzi równoległej przez Mateusza pokazano na zrzucie ekranu 3.9 (s. 16).

```
PS C:\Users\MATEUSZ\Desktop\zad2> git checkout -b rownolegla-Mateusz
Switched to a new branch 'rownolegla-Mateusz'
PS C:\Users\MATEUSZ\Desktop\zad2>
```

Rys. 3.9. Tworzenie gałęzi równoległej przez Mateusza

Commity implementujące usuwanie elementów na gałęzi Wiktora zaprezentowano na zrzucie ekranu 3.10 (s. 16).

```
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git checkout rownolegla-wiktor
M       BST.cpp
M       BST.h
Already on 'rownolegla-wiktor'
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git add BST.h
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git commit -m "Dodanie usuwania elementu"
[rownolegla-wiktor c779e17] Dodanie usuwania elementu
 1 file changed, 1 insertion(+)
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git add BST.cpp
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git commit -m "implementacja usuwania elementu"
[rownolegla-wiktor 98b7921] implementacja usuwania elementu
 1 file changed, 4 insertions(+)
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2>




PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git add main.cpp
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git commit -m "Dodanie usuniecia elementu do menu"
[rownolegla-wiktor 1f5e1be] Dodanie usuniecia elementu do menu
 1 file changed, 15 insertions(+), 6 deletions(-)
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2>
```

Rys. 3.10. Commity implementujące usuwanie elementów na gałęzi Wiktora

Wieloetapowe commity Mateusza na gałęzi równoległej zilustrowano na zrzucie ekranu 3.11 (s. 17).

```
PS C:\Users\MATEUSZ\Desktop\zad2> git status
On branch rownolegla-Mateusz
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   BST.h

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   BST.cpp
        modified:   Zad2.vcxproj.filters
        modified:   main.cpp

        BSTFileManager.cpp
        BSTFileManager.h
PS C:\Users\MATEUSZ\Desktop\zad2> git commit -m "Modyfikacja BST.h"
[rownolegla-Mateusz e5f881b] Modyfikacja BST.h
 1 file changed, 5 insertions(+), 1 deletion(-)
PS C:\Users\MATEUSZ\Desktop\zad2> git add BSTFileManager.h
PS C:\Users\MATEUSZ\Desktop\zad2> git commit -m "Utworzenie BSTFileManager.h"
[rownolegla-Mateusz fb27e0a] Utworzenie BSTFileManager.h
 1 file changed, 16 insertions(+)
 create mode 100644 BSTFileManager.h
PS C:\Users\MATEUSZ\Desktop\zad2> git commit -m "Modyfikacja BST.cpp"
[rownolegla-Mateusz a83f990] Modyfikacja BST.cpp
PS C:\Users\MATEUSZ\Desktop\zad2> git add BSTFileManager.cpp
PS C:\Users\MATEUSZ\Desktop\zad2> git commit -m "Utworzenie BSTFileManager.cpp"
[rownolegla-Mateusz 3f32a15] Utworzenie BSTFileManager.cpp
 1 file changed, 59 insertions(+)
 create mode 100644 BSTFileManager.cpp
PS C:\Users\MATEUSZ\Desktop\zad2> git add main.cpp
PS C:\Users\MATEUSZ\Desktop\zad2> git commit -m "Modyfikacja main.cpp"
[rownolegla-Mateusz fe69b7d] Modyfikacja main.cpp
 1 file changed, 65 insertions(+), 12 deletions(-)
On branch rownolegla-Mateusz
Changes not staged for commit:
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Zad2.vcxproj
        modified:   Zad2.vcxproj.filters

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\MATEUSZ\Desktop\zad2> git add .
PS C:\Users\MATEUSZ\Desktop\zad2> git commit -m "Abc"
[rownolegla-Mateusz 1e98045] Abc
 2 files changed, 8 insertions(+)
PS C:\Users\MATEUSZ\Desktop\zad2> git push origin rownolegla-Mateusz
```

Rys. 3.11. Wieloetapowe commity Mateusza na gałęzi równoległej

Commitowanie oraz scalanie na gałęzi Wiktora wraz z plikami dokumentacji pokazano na zrzucie ekranu 3.12 (s. 18).

```
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git checkout Wiktor
M       main.cpp
Switched to branch 'Wiktor'
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git add .
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git commit -m "Commit na gałęzi Wiktor"
[Wiktor 35009b5] Commit na gałęzi Wiktor
 4 files changed, 1 insertion(+), 1 deletion(-)
 create mode 100644 DOK/zdjecia/W1.png
 create mode 100644 DOK/zdjecia/W2.png
 create mode 100644 DOK/zdjecia/W3.png
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git push origin Wiktor
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 68.40 KiB | 17.10 MiB/s, done.
Total 8 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/WiktorMatusik/Zad2.git
   8cd035a..35009b5  Wiktor -> Wiktor
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git merge --no-ff Wiktor -m "Scalenie gałęzi Wiktor."
Merge made by the 'ort' strategy.
 DOK/zdjecia/W1.png | Bin 0 -> 4421 bytes
 DOK/zdjecia/W2.png | Bin 0 -> 28584 bytes
 DOK/zdjecia/W3.png | Bin 0 -> 44433 bytes
 main.cpp           |   2 +-
 4 files changed, 1 insertion(+), 1 deletion(-)
 create mode 100644 DOK/zdjecia/W1.png
 create mode 100644 DOK/zdjecia/W2.png
 create mode 100644 DOK/zdjecia/W3.png
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git push origin master
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 235 bytes | 235.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/WiktorMatusik/Zad2.git
   e9cc43b..a57d360  master -> master
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git status
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2>
```

Rys. 3.12. Commitowanie oraz scalanie na gałęzi Wiktora wraz z plikami dokumentacji

Pracę na gałęzi równoległej Mateusza i zatwierdzanie zmian zaprezentowano na zrzucie ekranu 3.13 (s. 18).

```
PS C:\Users\MATEUSZ\Desktop\zad2> git push origin rownolegla-Mateusz
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 6 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 1.10 KiB | 1.10 MiB/s, done.
Total 9 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 4 local objects.
remote:                                                              .h"
remote: Create a pull request for 'rownolegla-Mateusz' on GitHub by visiting:
remote:      https://github.com/WiktorMatusik/Zad2/pull/new/rownolegla-Mateusz
remote:
To https://github.com/WiktorMatusik/Zad2.git
 * [new branch]      rownolegla-Mateusz -> rownolegla-Mateusz
PS C:\Users\MATEUSZ\Desktop\zad2>


                            git add main.cpp
PS C:\Users\MATEUSZ\Desktop\zad2> git commit -m "Dodanie case z removeElement w main.cpp"
[rownolegla-Mateusz a78bfe9] Dodanie case z removeElement w main.cpp
 1 file changed, 9 insertions(+)
PS C:\Users\MATEUSZ\Desktop\zad2>
```

Rys. 3.13. Praca na gałęzi równoległej Mateusza i zatwierdzanie zmian

Dodawanie klas obsługi plików w gałęzi Wiktora zilustrowano na zrzucie ekranu

3.14 (s. 19).

```
no changes added to commit (use "git add" and/or "git commit -a")
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git add .\BSTFileManager.h
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git commit -m "Dodanie BSTFileManager.h"
[rownolegla-wiktor a22d2b9] Dodanie BSTFileManager.h
 1 file changed, 16 insertions(+)
 create mode 100644 BSTFileManager.h
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git add .\BSTFileManager.cpp
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git commit -m "Implementacja BSTFileManager"
[rownolegla-wiktor 182a61c] Implementacja BSTFileManager
 1 file changed, 59 insertions(+)
 create mode 100644 BSTFileManager.cpp
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git add .\main.cpp
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git commit -m "Dodanie funckji BSTFileManager do menu"
[rownolegla-wiktor 42f13b9] Dodanie funckji BSTFileManager do menu
 1 file changed, 34 insertions(+)
```

Rys. 3.14. Dodawanie klas obsługi plików w gałęzi Wiktora

Finalizację i wypchnięcie gałęzi równoległej Wiktora przedstawiono na zrzucie ekranu 3.15 (s. 19).

```
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git push origin rownolegla-wiktor
Enumerating objects: 60, done.
Counting objects: 100% (54/54), done.
Delta compression using up to 16 threads
Compressing objects: 100% (35/35), done.
Writing objects: 100% (37/37), 151.92 KiB | 12.66 MiB/s, done.
Total 37 (delta 17), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (17/17), completed with 7 local objects.
To https://github.com/WiktorMatusik/Zad2.git
   1f5e1be..d0ccc33  rownolegla-wiktor -> rownolegla-wiktor
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git checkout master
Switched to branch 'master'
Your branch is behind 'origin/master' by 4 commits, and can be fast-forwarded.
  (use "git pull" to update your local branch)
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git merge --no-ff rownolegla-wiktor -m "Scalenie rownolegla-wiktor do master"
Merge made by the 'ort' strategy.
 BST.cpp              |   1 -
 BST.h                |   2 ++
 BSTFileManager.cpp   |  59 +++++++
```

```
PS C:\Users\MATEUSZ\Desktop\zad2> git push origin rownolegla-Mateusz
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 6 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 1.10 KiB | 1.10 MiB/s, done.
Total 9 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 4 local objects.
remote:
remote: Create a pull request for 'rownolegla-Mateusz' on GitHub by visiting:
remote:      https://github.com/WiktorMatusik/Zad2/pull/new/rownolegla-Mateusz
remote:
To https://github.com/WiktorMatusik/Zad2.git
 * [new branch]      rownolegla-Mateusz -> rownolegla-Mateusz
PS C:\Users\MATEUSZ\Desktop\zad2>
```

Rys. 3.16. Wypchnięcie gałęzi Mateusza na GitHub

Finalizację prac i wysłanie zmian po stronie Mateusza zilustrowano na zrzucie ekranu 3.17 (s. 20).

```
PS C:\Users\MATEUSZ\Desktop\zad2> git push origin rownolegla-Mateusz
Enumerating objects: 25, done.
Counting objects: 100% (25/25), done.
Delta compression using up to 6 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (19/19), 3.83 KiB | 979.00 KiB/s, done.
Total 19 (delta 9), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (9/9), completed with 4 local objects.
PS C:\Users\MATEUSZ\Desktop\zad2> git checkout master
Your branch is up to date with 'origin/master'.
PS C:\Users\MATEUSZ\Desktop\zad2> git merge --no-ff rownolegla-Mateusz -m "Scalenie rownolegla-Mateusz do master"
Auto-merging BST.h
Auto-merging main.cpp
CONFLICT (content): Merge conflict in main.cpp
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\MATEUSZ\Desktop\zad2> git add .
PS C:\Users\MATEUSZ\Desktop\zad2> git commit -m "naprawa konfliktow"
[master 821c108] naprawa konfliktow
PS C:\Users\MATEUSZ\Desktop\zad2> git push origin master
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 6 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 356 bytes | 356.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/WiktorMatusik/Zad2.git
   dbf6bc0..821c108  master -> master
PS C:\Users\MATEUSZ\Desktop\zad2>
```

Rys. 3.17. Finalizacja prac i push po stronie Mateusza

Podczas scalania gałęzi równoległych z gałęzią główną (`master`), w wyniku modyfikacji tych samych fragmentów kodu w plikach projektowych, celowo wygenerowano wymagane konflikty. Zgodnie z założeniami, członkowie zespołu rozwiązali łącznie 6 konfliktów (po 3 przypadki na osobę).

Wykrycie i rozwiązanie 3 konfliktów w kodzie przez Wiktora przedstawiono na zrzucie ekranu 3.18 (s. 21).

```
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git checkout rownolegla-wiktor
Switched to branch 'rownolegla-wiktor'
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git pull origin master
remote: Enumerating objects: 31, done.
remote: Counting objects: 100% (29/29), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 21 (delta 11), reused 20 (delta 10), pack-reused 0 (from 0)
Unpacking objects: 100% (21/21), 3.19 KiB | 59.00 KiB/s, done.
From https://github.com/WiktorMatusik/Zad2
 * branch            master     -> FETCH_HEAD
   dbf6bc0..821c108  master     -> origin/master
Auto-merging BSTFileManager.cpp
CONFLICT (add/add): Merge conflict in BSTFileManager.cpp
Auto-merging BSTFileManager.h
CONFLICT (add/add): Merge conflict in BSTFileManager.h
Auto-merging main.cpp
CONFLICT (content): Merge conflict in main.cpp
Automatic merge failed; fix conflicts and then commit the result.
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git add .
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git commit -m "Wiktor: rozwiazano 3 konflikty"
[rownolegla-wiktor 315e9c0] Wiktor: rozwiazano 3 konflikty
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git checkout master
Switched to branch 'master'
Your branch and 'origin/master' have diverged,
and have 6 and 7 different commits each, respectively.
  (use "git pull" if you want to integrate the remote branch with yours)
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git pull origin master
From https://github.com/WiktorMatusik/Zad2
 * branch            master     -> FETCH_HEAD
Auto-merging BSTFileManager.cpp
CONFLICT (add/add): Merge conflict in BSTFileManager.cpp
Auto-merging BSTFileManager.h
CONFLICT (add/add): Merge conflict in BSTFileManager.h
Auto-merging main.cpp
CONFLICT (content): Merge conflict in main.cpp
Automatic merge failed; fix conflicts and then commit the result.
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git merge --no-ff rownolegla-wiktor -m "Finalne scalenie Wiktora"
error: Merging is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git add .
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git commit -m "mergowanie"
[master 6a75cb0] mergowanie
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git merge --no-ff rownolegla-wiktor -m "Finalne scalenie Wiktora"
Auto-merging BSTFileManager.cpp
CONFLICT (content): Merge conflict in BSTFileManager.cpp
Auto-merging BSTFileManager.h
```

Rys. 3.18. Wykrycie i rozwiązanie 3 konfliktów w kodzie przez Wiktora

Kroki końcowe scalania i operację push po rozwiązaniu konfliktów Wiktora pokazano na zrzucie ekranu 3.19 (s. 21).

```
Automatic merge failed; fix conflicts and then commit the result.
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git merge --no-ff rownolegla-wiktor -m "Finalne scalenie Wiktora"
error: Merging is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git add .
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git commit -m "mergowanie"
[master 6a75cb0] mergowanie
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git merge --no-ff rownolegla-wiktor -m "Finalne scalenie Wiktora"
Auto-merging BSTFileManager.cpp
CONFLICT (content): Merge conflict in BSTFileManager.cpp
Auto-merging BSTFileManager.h
CONFLICT (content): Merge conflict in BSTFileManager.h
Auto-merging main.cpp
CONFLICT (content): Merge conflict in main.cpp
Automatic merge failed; fix conflicts and then commit the result.
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2> git push origin master
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 1.02 KiB | 1.02 MiB/s, done.
Total 6 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/WiktorMatusik/Zad2.git
   821c108..6a75cb0  master -> master
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad2>
```

Rys. 3.19. Kroki końcowe scalania i push po rozwiązaniu konfliktów Wiktora

Wystąpienie konfliktów podczas scalania gałęzi Mateusza zilustrowano na zrzucie ekranu 3.20 (s. 22).

```
Your branch is up to date with 'origin/master'.
PS C:\Users\MATEUSZ\Desktop\zad2> git checkout master
Already on 'master'
Your branch is up to date with 'origin/master'.
PS C:\Users\MATEUSZ\Desktop\zad2> git pull origin master
remote: Enumerating objects: 14, done.
remote: Counting objects: 100% (14/14), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 10 (delta 8), reused 8 (delta 6), pack-reused 0 (from 0)
Unpacking objects: 100% (10/10), 1.15 KiB | 24.00 KiB/s, done.
From https://github.com/WiktorMatusik/Zad2
 * branch            master     -> FETCH_HEAD
   a57d360..62ea59d  master     -> origin/master
Updating a57d360..62ea59d
Fast-forward
 BST.cpp  |  4 ++++
 BST.h    |  1 +
 main.cpp | 21 +++++++++++++++++------
 3 files changed, 20 insertions(+), 6 deletions(-)
PS C:\Users\MATEUSZ\Desktop\zad2> git merge --no-ff rownolegla-Mateusz
Auto-merging BST.cpp
CONFLICT (content): Merge conflict in BST.cpp
Auto-merging BST.h
CONFLICT (content): Merge conflict in BST.h
Auto-merging main.cpp
CONFLICT (content): Merge conflict in main.cpp
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\MATEUSZ\Desktop\zad2>
```

Rys. 3.20. Wystąpienie konfliktów podczas scalania gałęzi Mateusza

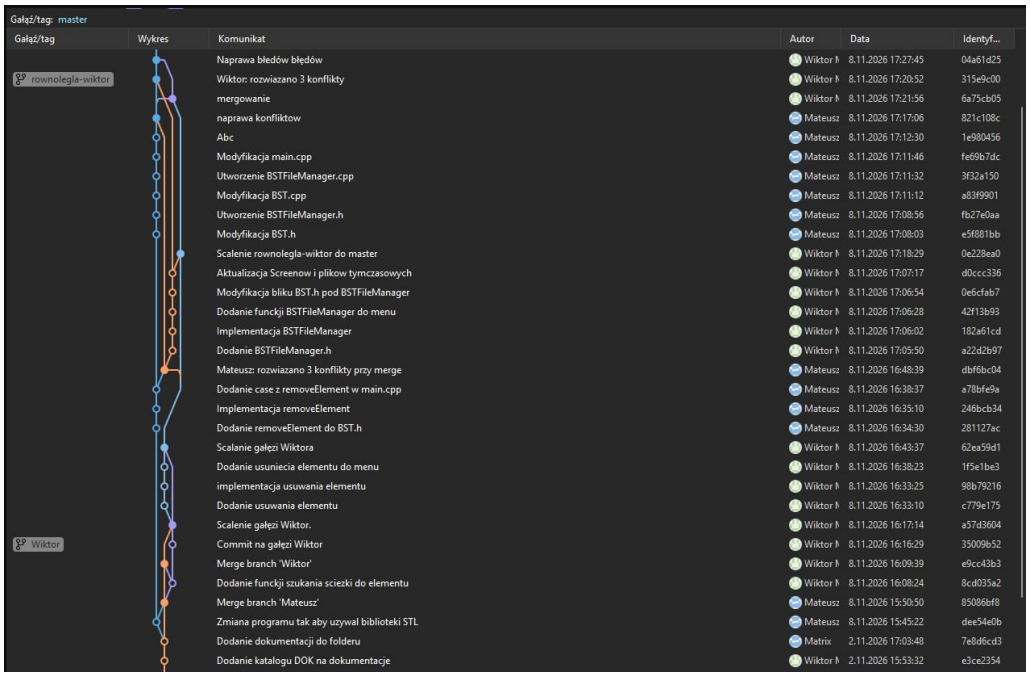
Rozwiązanie 3 konfliktów przez Mateusza oraz zatwierdzenie scalenia zaprezentowano na zrzucie ekranu 3.21 (s. 22).

```
PS C:\Users\MATEUSZ\Desktop\zad2> git add .
PS C:\Users\MATEUSZ\Desktop\zad2> git commit -m "Mateusz: rozwiazano 3 konflikty przy merge"
[master dbf6bc0] Mateusz: rozwiazano 3 konflikty przy merge
PS C:\Users\MATEUSZ\Desktop\zad2> git push origin master
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 6 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 338 bytes | 338.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/WiktorMatusik/Zad2.git
   62ea59d..dbf6bc0  master -> master
PS C:\Users\MATEUSZ\Desktop\zad2>
```

Rys. 3.21. Rozwiązanie 3 konfliktów przez Mateusza i zatwierdzenie scalenia

Proces ten zakończył się pełną integracją kodu, co potwierdzono w podsumowaniach statusu repozytorium oraz weryfikacji na platformie GitHub.

Graficzną reprezentację drzewa commitów, historii gałęzi oraz pracy zespołu, pobraną bezpośrednio z repozytorium na platformie GitHub, zaprezentowano na zrzucie 3.22 (s. 23).



Rys. 3.22. Drzewo gałęzi i commitów w repozytorium GitHub

4. Implementacja

4.1. Implementacja algorytmu

Projekt został zaimplementowany w języku C++ z podziałem na modularne pliki nagłówkowe oraz źródłowe zgodnie z wymaganiami architektonicznymi. Główna logika struktury danych została zamknięta w klasie `BST`, operacje plikowe w klasie `BSTFileManager`, a interfejs użytkownika zrealizowano w module głównym `main.cpp`.

Wewnętrznie klasa `BST` przechowuje dane w kontenerze biblioteki standardowej STL `std::set<int>`, co gwarantuje unikalność oraz automatyczny porządek elementów. Ponieważ kontener ten nie jest klasycznym drzewem wskaźnikowym, do celów graficznego wyświetlania oraz wyszukiwania ścieżki zrealizowano algorytm dynamicznego budowania tymczasowego, zbalansowanego drzewa binarnego na bazie wektora.

4.2. Ciekawe fragmenty kodu

W trakcie realizacji projektu zaimplementowano szereg rozwiązań programistycznych, najciekawsze to między innymi:

- Dynamiczne budowanie zbalansowanego drzewa pomocniczego W BST.cpp 1 (s. 24) w klasie `BST` dane przechowywane są w kontenerze `std::set<int>`. Ponieważ kontener ten ukrywa wskaźnikową strukturę węzłów przed bezpośrednim dostępowaniem, na potrzeby operacji takich jak wyszukiwanie ścieżki czy wyświetlanie graficzne zaimplementowano algorytm `buildBalancedTree`. Pozwala on na rekurencyjne zbudowanie idealnie zbalansowanego tymczasowego drzewa binarnego na podstawie posortowanej tablicy w czasie $O(n)$:

```
1 BST::Node* BST::buildBalancedTree(const int* arr, int start,
      int end) const {
2     if (start > end) return nullptr;
3     int mid = start + (end - start) / 2;
4     Node* node = new Node(arr[mid]);
5     node->left = buildBalancedTree(arr, start, mid - 1);
6     node->right = buildBalancedTree(arr, mid + 1, end);
7     return node;
8 }
```

Listing 1. Rekurencyjne budowanie zbalansowanego drzewa BST w BST.cpp

- Wyszukiwanie ścieżki do zadanego elementu W BST.cpp 2 (s. 25) metoda `findPath` wykorzystuje zbalansowane drzewo pomocnicze do wyznaczenia peł-

nej ścieżki od korzenia do szukanego węzła. Wynik ścieżki zapisywany jest w przekazanym przez referencję wektorze `std::vector<int>`. Zwracana wartość logiczna określa, czy dany element w ogóle istnieje w strukturze, a w przypadku braku – wektor jest czyszczony, a utworzone drzewo pomocnicze bezpiecznie zwalniane w pamięci (`destroyTree`):

```
bool BST::findPath(int target, std::vector<int>& path) const {
    path.clear();
    if (treeData.empty()) {
        return false;
    }

    vector<int> vec(treeData.begin(), treeData.end());
    Node* tempRoot = buildBalancedTree(vec.data(), 0,
   static_cast<int>(vec.size()) - 1);

    Node* current = tempRoot;
    bool found = false;

    while (current != nullptr) {
        path.push_back(current->data);

        if (current->data == target) {
            found = true;
            break;
        }
        else if (target < current->data) {
            current = current->left;
        }
        else {
            current = current->right;
        }
    }

    if (!found) {
        path.clear();
    }

    destroyTree(tempRoot);

    return found;
}
```

Listing 2. Wyszukiwanie ścieżki do elementu w BST.cpp

- Binarna persystencja danych W BSTFileManager.cpp 3 (s. 26) w klasie `BSTFileManager` zastosowano strumienie binarne (`std::ofstream` w trybie `ios::binary`) do bezpośredniego zapisu stanu drzewa. Najpierw do pliku zapisywana jest liczba elementów (`size_t`), a następnie surowe wartości bajtowe poszczególnych liczb całkowitych z kontenera:

```
bool BSTFileManager::saveToBinaryFile(const BST& tree, const
    string& filename) {
     ofstream file(filename, ios::binary);
     if (!file.is_open()) return false;

     const auto& data = tree.getData();
     size_t count = data.size();

     file.write(reinterpret_cast<const char*>(&count), sizeof(
    count));

     for (int val : data) {
         file.write(reinterpret_cast<const char*>(&val), sizeof(
    val));
     }

     file.close();
     return true;
}
```

Listing 3. Binarny zapis drzewa BST do pliku w BSTFileManager.cpp

4.3. Opis powstałych wyników

W wyniku prac nad programem uzyskano w pełni działającą aplikację konsolową spełniającą wszystkie kryteria funkcjonalne i architektoniczne:

- **Działanie interfejsu i menu:** Program uruchamia pętlę główną z interaktywnym menu tekstowym, które pozwala użytkownikowi na płynne wybieranie operacji (dodawanie, usuwanie, czyszczenie, wyszukiwanie ścieżek, wyświetlanie oraz operacje plikowe) z zabezpieczeniem przed niepoprawnymi danymi.
- **Wizualizacja i przejścia:** Użytkownik może wywołać graficzną prezentację drzewa w konsoli oraz wypisać elementy zgodnie z algorytmami *Preorder*, *Inorder* i *Postorder*.
- **Persystencja danych:** Opracowany moduł plikowy poprawnie generuje pliki

binarne stanu drzewa oraz potrafi wczytywać dane z zewnętrznych plików tekstowych (.txt) zarówno do pustego, jak i do już istniejącego drzewa.

- **Artefakty projektowe i wersjonowanie:** W ramach prac zespołowych poprawnie zrealizowano gałęziowanie w systemie Git (praca sekwencyjna i równoległa), zatwierdzono wymaganą liczbę commitów przez obu członków grupy oraz rozwiązano 6 zaplanowanych konfliktów scalania. Dodatkowo na podstawie skomentowanego kodu źródłowego wygenerowano komplet dokumentacji technicznej za pomocą narzędzia Doxygen.

5. Wnioski

Realizacja projektu pozwoliła na praktyczne zastosowanie zaawansowanych mechanizmów języka C++ oraz kontenerów biblioteki standardowej STL w implementacji struktury drzewa poszukiwań binarnych (BST). Stworzona aplikacja spełnia wszystkie założone wymagania funkcjonalne, oferując intuicyjne menu konsolowe, kompleksowe operacje na węzłach, wyszukiwanie ścieżek, różnorodne metody przechodzenia struktury (preorder, inorder, postorder) oraz niezawodną persystencję danych w plikach tekstowych i binarnych. Niezwykle istotnym aspektem przedsięwzięcia było również opanowanie pracy w dwuosobowym zespole z wykorzystaniem systemów kontroli wersji Git i GitHub. Przeprowadzone scenariusze pracy równoległej, scalania gałęzi oraz celowego rozwiązywania konfliktów znacząco wpłynęły na poszerzenie kompetencji z zakresu inżynierii oprogramowania. Opracowany program, połączony z automatycznie wygenerowaną przez Doxygen dokumentacją techniczną, stanowi spójne i w pełni funkcjonalne narzędzie informatyczne.

Bibliografia

[1] *Strona internetowa o drzewach BST*. URL: https://binarnie.pl/drzewo-binarne/ (dostęp 28.10.2026).

[2] *Strona internetowa Inorder, Preorder oraz Postorder drzew BST*. URL: https://algodaily.com/sections/trees-interview-questions (dostęp 2.11.2026).

Spis rysunków

2.1. Drzewo BST 5
2.2. Drzewo BST Inorder 6
2.3. Drzewo BST Preorder 7
2.4. Drzewo BST Postorder 7
3.1. Tworzenie nowej gałęzi w interfejsie graficznym Visual Studio 12
3.2. Zatwierdzanie zmian i wypychanie gałęzi Mateusz 13
3.3. Scalanie gałęzi Mateusz do gałęzi master 14
3.4. Widok scalonej gałęzi w serwisie GitHub 14
3.5. Tworzenie nowej gałęzi przez Wiktora 15
3.6. Sprawdzanie statusu i dodawanie commitów na gałęzi Wiktor 15
3.7. Wypchnięcie gałęzi Wiktor i scalenie jej z masterem 15
3.8. Pobranie najnowszych zmian i tworzenie gałęzi równoległej przez Wiktora 16
3.9. Tworzenie gałęzi równoległej przez Mateusza 16
3.10. Commity implementujące usuwanie elementów na gałęzi Wiktora . . 16
3.11. Wieloetapowe commity Mateusza na gałęzi równoległej 17
3.12. Commitowanie oraz scalanie na gałęzi Wiktora wraz z plikami dokumentacji 18
3.13. Praca na gałęzi równoległej Mateusza i zatwierdzanie zmian 18
3.14. Dodawanie klas obsługi plików w gałęzi Wiktora 19
3.15. Finalizacja i push gałęzi równoległej Wiktora 19
3.16. Wypchnięcie gałęzi Mateusza na GitHub 20
3.17. Finalizacja prac i push po stronie Mateusza 20
3.18. Wykrycie i rozwiązanie 3 konfliktów w kodzie przez Wiktora 21
3.19. Kroki końcowe scalania i push po rozwiązaniu konfliktów Wiktora . . 21
3.20. Wystąpienie konfliktów podczas scalania gałęzi Mateusza 22
3.21. Rozwiązanie 3 konfliktów przez Mateusza i zatwierdzenie scalenia . . 22
3.22. Drzewo gałęzi i commitów w repozytorium GitHub 23

Spis tabel

Spis listingów

1. Rekurencyjne budowanie zbalansowanego drzewa BST w BST.cpp . . 24
2. Wyszukiwanie ścieżki do elementu w BST.cpp 25
3. Binarny zapis drzewa BST do pliku w BSTFileManager.cpp 26